

Fundamentos de Aprendizagem Automática

Class 03 — Linear Models

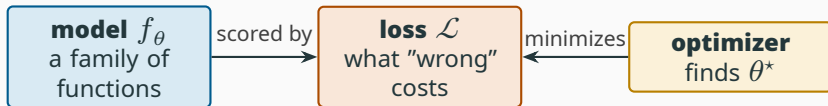
Mário Antunes

Universidade de Aveiro

2026

From Optimization to Models

Where We Left Off



Class 02: the optimizers (gradient descent, JAX). Class 03: the first *models* and their *losses*

- Class 01: a good model **compresses** the data; we measure it on **unseen** data
- Class 02: training is $\theta^* = \arg \min_{\theta} \mathcal{L}_{data}(\theta) + \lambda \Omega(\theta)$
- Today: three classic models, each one *a choice of f_θ and $\mathcal{L}$*

Three Models, One Recipe

Model	Predicts	Loss	Fit by
Linear regression	a number	squared error	closed form or GD
Perceptron	a class	$\max(0, -y f(x))$	online update
Logistic regression	a probability	cross-entropy	GD (no closed form)

- All three are trained with **jax.grad** in the lab: we write the loss, JAX gives the gradient
- Polynomial features and regularization can be added to any of them
- Every one is compared with its **scikit-learn** twin: same split, same scaling, a printed difference
- Next class: the **probabilistic** view of the same models, and Naive Bayes

Two Datasets

Spam detection

(classification)

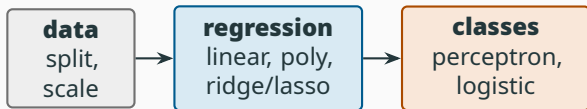
- Spambase: **4601** e-mails, **57** features
- Word and character frequencies, capital runs
- **39.4%** spam
- Most values are 0, a few huge
- 571 duplicate rows

- Real, noisy, small: every demo runs in seconds (data/* .parquet, read with polars)

House prices (regression)

- California housing: **20640** districts, **8** features
- Income, age, rooms, occupancy, latitude, longitude
- Target: median value (100 000 USD)
- 992 districts capped at 5.0
- Up to 1243 people per house

Map of the Class



evaluation: regression metrics (MSE, MAE, sMAPE, R^2)
· classification metrics (CM, precision, recall, F1, MCC)

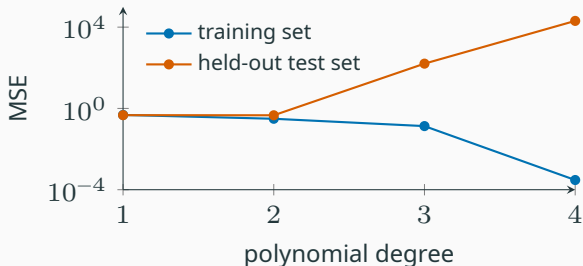
- **Lecture (1 h):** models, losses, metrics, splits, regularization
- **Demos:** 01_linear_regression, 02_perceptron_logistic (notebooks R1–R5, C1–C4)
- **Lab (2 h):** build the models in JAX, compare each with scikit-learn

Project 1 Is Released

- **Tribal benchmark:** compare models from the five tribes on classification and regression tasks
- Today's models: the perceptron is the seed of the **Connectionist** tribe; next class covers the **Bayesians**
- The evaluation protocol of this class (splits, scaling, metrics, regularization) is what the report will be judged on
- Specification: `projects/project01.pdf`; due at Class 08

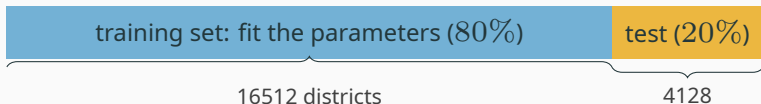
Splitting and Preparing Data

Why Hold Out Data?

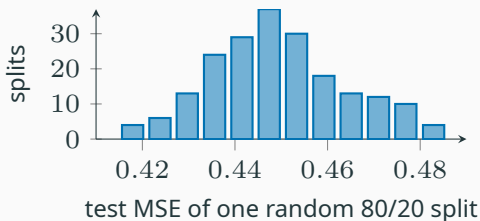


- House prices, 300 training examples, polynomial degrees 1 to 4 (demo R4)
- The **training** error says “perfect”; only **unseen** data shows the truth
- Every number we report must come from data the model did not train on

Hold-Out: One Split



- Shuffle, then cut: the model never sees the test rows while training
- Cheap, but the estimate depends on **which** rows landed in the test set



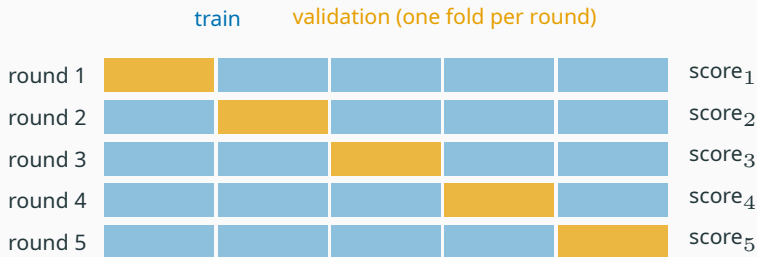
One Split Is Not Enough

- The same linear model on **200 random splits**: test MSE from **0.415** to **0.486** (mean 0.449, std 0.015)
- With 4128 test rows the spread is small; with a few hundred rows it would be large
- **Repeated hold-out** and **cross-validation** turn one number into a mean *and* a spread

Protocol	Mean test MSE	Spread
one 80/20 split	0.449	± 0.015 over 200 splits
5-fold CV	0.449	std of folds 0.019, 95% CI ± 0.024
10-fold CV	0.449	std of folds 0.028, 95% CI ± 0.020

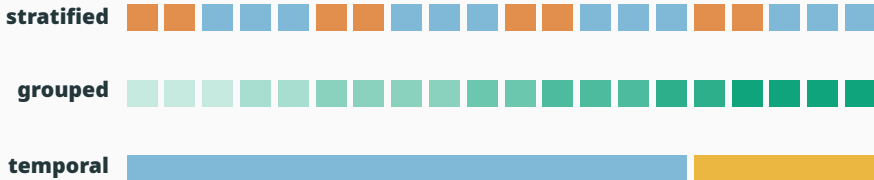
- Class 01: confidence intervals, corrected resampled *t*-test, McNemar

k-Fold Cross-Validation



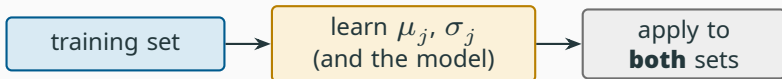
- Every example is used for validation **exactly once**; report the mean and the spread of the k scores
- Use it to **choose** hyper-parameters (degree, λ , threshold): a separate **test** set is touched once, at the end

Stratified, Grouped and Temporal Splits



- **Stratified:** keeps the class proportions; on 100 e-mails the spam fraction of train and test differs by **0.092** on average with a random split, **0.015** stratified (lab A1)
- **Grouped:** correlated rows must stay together, or the test set leaks
- **Temporal:** never test on data older than the training data

Scale With the Training Set Only

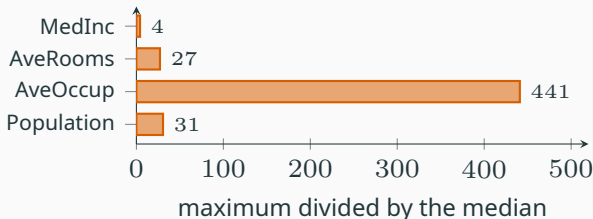


wrong: μ_j, σ_j from all the data
 $\Rightarrow$ the test set shapes the model

$$z_j = \frac{x_j - \mu_j}{\sigma_j}, \quad \mu_j, \sigma_j \text{ estimated on the training rows only}$$

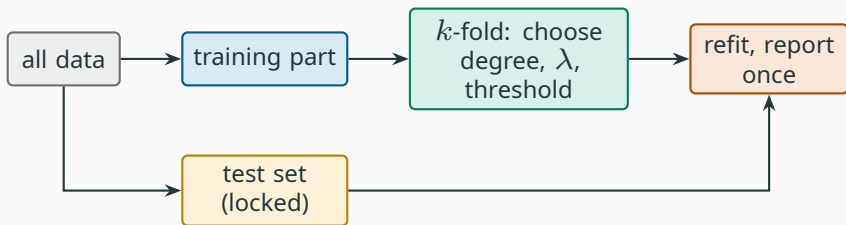
- Same 1000 steps: standardized housing loss **0.455**, raw features **0.532** (conditioning, Class 02)
- Any statistic learned from data (mean, vocabulary, bins, threshold) stays **inside** the training part (data leakage, Class 01)

Skewed Features: Take a Logarithm



- A few districts with 1200+ people per house would dominate a linear fit and, after polynomial expansion, explode
- Housing: log of AveRooms, AveBedrms, Population, AveOccup
- Spam: $\log(1 + x)$ of all 57 frequencies (most are exactly 0), then standardize
- Transformations are **choices made by us** and chosen without looking at the test set

The Protocol Used All Class



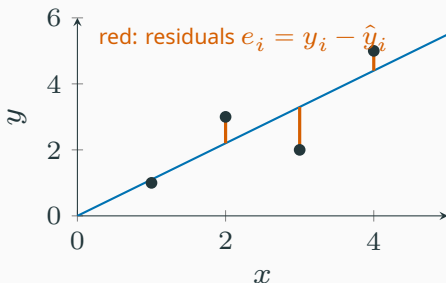
- Split once (stratified for classes), keep the test set **locked**
- Fit scalers and models on training folds only; choose hyper-parameters on validation folds
- Refit on the whole training part with the chosen values; evaluate on the test set **once**
- Report the number **with** its baseline and its spread

Live Demo R1: Splitting the Data

- Notebook `01_linear_regression.ipynb`, section **R1**
- 200 random hold-out splits, 5-fold and 10-fold cross-validation, and their spreads
- Standardization computed on the training part only
- Question: what would change if the file were sorted by latitude?

Linear Regression

The Model: a Hyperplane

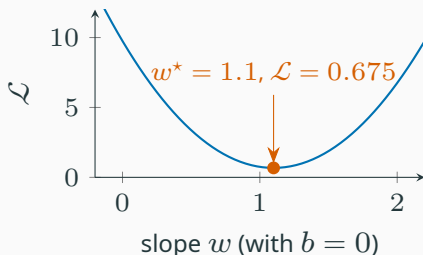


$$\hat{y} = w^\top x + b = w_1 x_1 + \dots + w_d x_d + b$$

- d weights and one bias: $d + 1$ numbers **are** the model
- Each weight: the change of $\hat{y}$ per unit of feature j , the others fixed
- Housing: $d = 8$; with standardized inputs the weights are comparable

The Loss: Mean Squared Error

$$\mathcal{L}(w, b) = \frac{1}{N} \sum_{i=1}^N (w^\top x_i + b - y_i)^2$$



- A **quadratic** in (w, b) : a convex bowl with one minimum (Class 02)
- Squaring: errors cannot cancel, large errors cost more
- Same four points as the previous slide

Worked Example: Least Squares by Hand

Points $(1, 1), (2, 3), (3, 2), (4, 5)$, so $\bar{x} = 2.5$ and $\bar{y} = 2.75$

$$w = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2} = \frac{5.5}{5} = 1.1, \quad b = \bar{y} - w\bar{x} = 0$$

x	y	$\hat{y}$	e
1	1	1.1	-0.1
2	3	2.2	0.8
3	2	3.3	-1.3
4	5	4.4	0.6

$$\text{MSE} = \frac{0.01 + 0.64 + 1.69 + 0.36}{4} = 0.675$$

The Gradient and the Normal Equation

$$\nabla_w \mathcal{L} = \frac{2}{N} X^\top (Xw + b - y) \quad \frac{\partial \mathcal{L}}{\partial b} = \frac{2}{N} \sum_i (\hat{y}_i - y_i)$$

- Gradient = features $\times$ residuals: each weight moves to explain the residual it can see
- Set it to zero. With $A = [X \ \mathbf{1}]$ and $\theta = (w, b)$:

$$A^\top A \theta = A^\top y \implies \theta^\star = (A^\top A)^{-1} A^\top y$$

- The **normal equations**: a linear system. Solve it (`np.linalg.solve`), never invert
- Exists whenever $A^\top A$ is invertible: no perfectly collinear features, and $N \geq d + 1$
- In the lab: **you never type this gradient**; `jax.grad` of the loss gives it

Worked Example: Gradient Descent

Same four points, start $w = b = 0, \eta = 0.05$

step	w	b	$\mathcal{L}$	$\partial_w \mathcal{L}$	$\partial_b \mathcal{L}$
0	0.000	0.000	9.750	-16.500	-5.500
1	0.825	0.275	0.940	-2.750	-0.825
2	0.963	0.316	0.699	-0.481	-0.055
3	0.987	0.319	0.692	-0.107	0.071

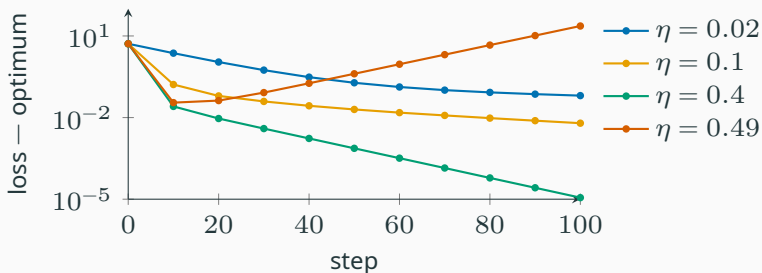
- Step 0:
 $\partial_w \mathcal{L} = \frac{2}{4} \sum_i (0 - y_i) x_i = -0.5 \cdot 33 = -16.5$, so
 $w_1 = 0 - 0.05 \cdot (-16.5) = 0.825$
- Heading for $(w, b) = (1.1, 0)$ and $\mathcal{L} = 0.675$: the answer of the closed form
- Linear regression is the one model where we can **check** the iterative answer against the exact one

Closed Form or Gradient Descent?

	Normal equation	Gradient descent
Cost	$O(Nd^2 + d^3)$ once	$O(Nd)$ per step, many steps
Needs Tuning	$A^T A$ invertible none	any differentiable loss learning rate, steps
Large d	hopeless (10^6 features)	works (mini-batches, Class 02)
Other losses (Huber, L1, ...)	no closed form	change one line

- For 8 housing features the closed form is instant; **all other models today have no closed form** (or only a variant)
- Both give the same weights: difference $6 \cdot 10^{-15}$ (normal equation vs scikit-learn), $2 \cdot 10^{-6}$ (300 GD steps)

The Learning Rate Comes from the Curvature



- The Hessian of the MSE is constant: $H = \frac{2}{N}A^{\top}A$.
Housing: eigenvalues **0.102** to **4.166**, condition number **41**
- Stable only for $\eta < 2/\lambda_{\max} = 0.48$: $\eta = 0.49$ diverges, $\eta = 0.02$ crawls along the flat direction (Class 02)

Linear Regression in JAX

```
import jax, jax.numpy as jnp

def predict(params, X):
    return X @ params["w"] + params["b"]

def mse_loss(params, X, y):
    return jnp.mean((predict(params, X) - y) ** 2)

params = {"w": jnp.zeros(8), "b": jnp.zeros(())}
grad = jax.jit(jax.grad(mse_loss))          # derived, not typed
for _ in range(300):
    g = grad(params, X_train, y_train)
    params = jax.tree.map(lambda p, gp: p - 0.4 * gp, params, g)
```

- Parameters are a **pytree** (a dictionary): `jax.grad` returns the gradient with the same structure
- Change the loss (add a penalty, use Huber) and **nothing else** changes: this is the lab's method

Why Squared Error? The Probabilistic View

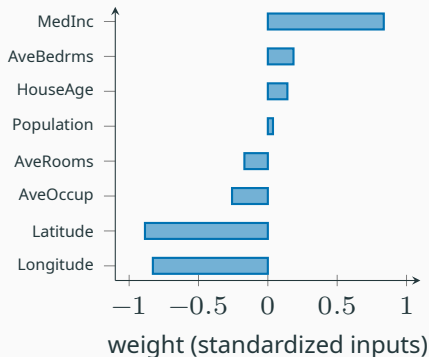
$$y = w^\top x + b + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2)$$

$$\implies p(y \mid x) = \mathcal{N}(w^\top x + b, \sigma^2)$$

$$-\log \prod_i p(y_i \mid x_i) = \frac{1}{2\sigma^2} \sum_i (y_i - \hat{y}_i)^2 + N \log \sigma + \text{const}$$

- Minimizing the MSE is **maximum likelihood** under Gaussian noise (Class 01: the loss is a code length)
- Laplace noise gives the absolute error (MAE, robust to outliers); Gaussian noise punishes outliers heavily
- The same idea gives **cross-entropy** for classes (a few slides ahead)

Result: House Prices



Test set (4128 districts)

	RMSE	R^2
mean baseline	1.148	0.000
linear	0.649	0.681

RMSE 0.649 = **64 900 USD**
typical error

- Income raises the price; latitude and longitude (the coast) dominate the rest
- About 32% of the variance is unexplained: what a hyperplane cannot see (and the cap at 5.0)

Live Demo R2: Linear Regression

- Notebook `01_linear_regression.ipynb`, section **R2**
- The normal equation, gradient descent with `jax.grad`, and scikit-learn: three routes to the same weights
- The Hessian of the loss, its eigenvalues, and four learning rates
- Predicted versus actual: the **cap at 5.0** is visible as a horizontal wall

Regression Metrics

How Wrong Are the Predictions?

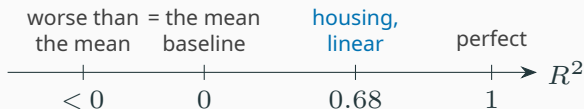
$$e_i = y_i - \hat{y}_i \quad (\text{the residual})$$

$$\text{MSE} = \frac{1}{n} \sum_i e_i^2 \quad \text{RMSE} = \sqrt{\text{MSE}} \quad \text{MAE} = \frac{1}{n} \sum_i |e_i|$$

- **MSE:** the loss we optimize; squared unit, punishes large errors
- **RMSE:** back in the unit of y , dominated by the large errors
- **MAE:** every error counts in proportion; robust to outliers
- Always **RMSE** $\geq$ **MAE**: the gap measures how uneven the errors are
- Lower is better; none is meaningful without a **baseline**

R^2 : Compared with Predicting the Mean

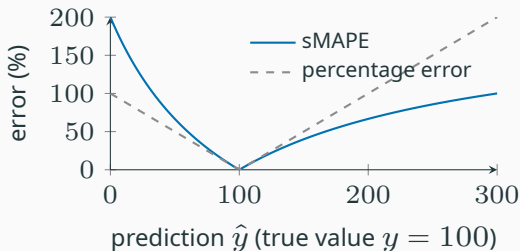
$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2} = 1 - \frac{\text{MSE of the model}}{\text{MSE of the mean baseline}}$$



- **Fraction of the variance explained** by the model; 0 is the mean baseline, 1 is perfect
- It can be **negative**: a constant prediction of 10 for values near 3 gives $R^2 = -98$
- Scale-free, but never enough alone: it says nothing about the size of the errors in USD

sMAPE: a Symmetric Relative Error

$$\text{sMAPE} = \frac{100}{n} \sum_i \frac{2 |y_i - \hat{y}_i|}{|y_i| + |\hat{y}_i|} \quad [\%]$$



- True value 100: prediction 50 gives **66.7%**, prediction 150 gives **40.0%** (same error, plain percentage error 50% for both)
- So it punishes **under-prediction** more; unstable near 0: use it for prices

Worked Example: the Four Metrics

Residuals of the least-squares line:

$$e = (-0.1, 0.8, -1.3, 0.6), \quad y = (1, 3, 2, 5), \quad \bar{y} = 2.75$$

Metric	Computation	Value
MSE	$(0.01 + 0.64 + 1.69 + 0.36)/4$	0.675
RMSE	$\sqrt{0.675}$	0.822
MAE	$(0.1 + 0.8 + 1.3 + 0.6)/4$	0.700
R^2	$1 - 2.7/8.75$	0.691
sMAPE	mean of $\frac{2 e_i }{y_i + \hat{y}_i}$: 9.5%, 30.8%, 49.1%, 12.8%	25.5%

- $\sum (y_i - \bar{y})^2 = 3.0625 + 0.0625 + 0.5625 + 5.0625 = 8.75$
- Four numbers, four questions: how big (RMSE), how typical (MAE), how much explained (R^2), how relative (sMAPE)

Metrics Disagree: Outliers

Predictions	MSE	RMSE	MAE	sMAPE	R^2
errors ± 0.1	0.010	0.100	0.100	3.5%	0.98
same, one error of +3	1.93	1.39	0.70	16.5%	-2.86

- True values 2, 2.5, 3, 3.5, 4; a single prediction 3 units off
- **MSE** $\times$ **193**, MAE $\times$ 7: squaring lets one point dominate
- Choose by the **cost of a mistake**: quadratic cost (safety margins) $\Rightarrow$ MSE/RMSE; linear cost, dirty data $\Rightarrow$ MAE
- Report at least one absolute and one relative measure, plus the baseline

Result: House Prices

Model	MSE	RMSE	MAE	sMAPE	R^2
mean baseline	1.319	1.148	0.904	44.6%	0.000
linear regression	0.421	0.649	0.472	25.6%	0.681

- Units: 100 000 USD. The typical error falls from **114 800** USD to **64 900** USD
- RMSE (0.649) above MAE (0.472): a few districts are badly wrong (the capped ones, the very expensive)
- sMAPE 25.6%: a quarter of the price, on average.
Useful, not precise: the next slides try to do better

Live Demo R3: Regression Metrics

- Notebook `01_linear_regression.ipynb`, section **R3**
- The five metrics on the test set, checked against `sklearn.metrics`
- One outlier: MSE explodes, MAE barely moves
- sMAPE asymmetry (50 versus 150) and a negative R^2
- Question: which metric would you show a house buyer?
A bank?

Polynomial Feature Expansion

Still Linear, in the Parameters

$$\hat{y} = w_0 + w_1x + w_2x^2 + w_3x^3 = w_0 + w^\top \phi(x)$$

$$\phi(x) = (x, x^2, x^3)$$

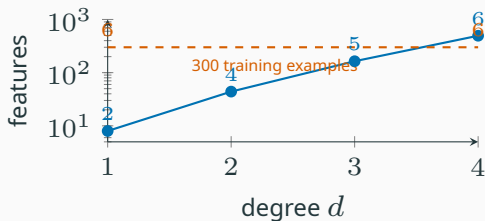
- The model is a **curve** in x but a **hyperplane** in the new features $\phi(x)$
- The loss is still a quadratic in w : the same normal equation, the same `jax.grad`, the same metrics
- With several inputs: **all monomials** up to degree d , e.g. degree 2 for (x_1, x_2) :

$$\phi(x_1, x_2) = (x_1, x_2, x_1^2, x_1x_2, x_2^2)$$

- The products x_1x_2 are **interactions**: the effect of one feature depends on another

The Number of Features Explodes

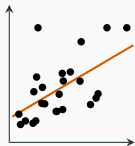
$$\# \text{features} = \binom{n + d}{d} - 1 \quad (n \text{ inputs, degree } d)$$



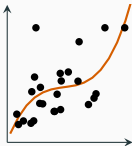
- The 8 housing features: 8, **44**, **164**, **494** columns (degrees 1 to 4)
- Beyond N columns the model **interpolates** the training set
- **Standardize after expanding** (x^4 has a very different scale)

Curves of Growing Flexibility

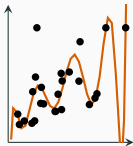
degree 1: underfits



degree 3: smooth



degree 9: memorizes



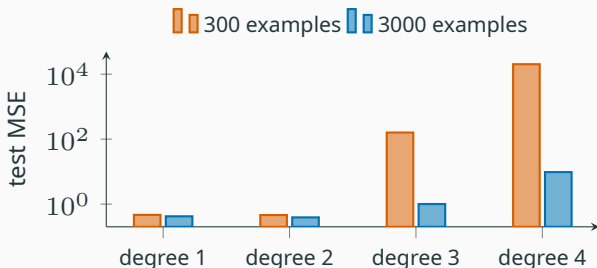
- 25 districts: value against (standardized) median income, fitted with polynomials of degree 1, 3 and 9
- Degree 1 misses the bend, degree 9 chases the noise and swings wildly at the edges

Training Error Versus Test Error

degree	features	train MSE	test MSE
1	8	0.478	0.467
2	44	0.310	0.460
3	164	0.135	159.8
4	494	0.0003	20 289

- 300 training examples of the 8 housing features; the **training** error falls at every degree
- At degree 4 there are more features than examples: the fit is perfect on the training set, useless on the test set
- Degree 3 is *worse* than degree 2 by a factor of **350**:
outlying districts, cubed, dominate
- The **test** (or validation) error picks the degree, never the training error

More Data Tames a Flexible Model



- Ten times more data: degree 3 goes from **160** to **1.0**, degree 4 from **20 000** to **9.7**
- Flexibility only pays when the data can *support* it: at 3000 examples degree 2 (**0.390**) finally beats the line (**0.421**)
- The other cure keeps the data and **constrains the model**: regularization

Live Demo R4: Polynomial Features

- Notebook `01_linear_regression.ipynb`, section **R4**
- Degrees 1, 3, 9, 15 on 25 points; the feature counts; train and test error for 300 and 3000 examples
- Question: why does the test error of degree 3 explode although the training error is small?

Regularization

Penalize Large Weights

$$\min_{w, b} \underbrace{\frac{1}{N} \sum_i (\hat{y}_i - y_i)^2}_{\text{fit the data}} + \lambda \underbrace{\Omega(w)}_{\text{keep the weights small}}$$

	$\Omega(w)$	Name	Effect
L_2	$\ w\ _2^2 = \sum_j w_j^2$	ridge	shrinks all weights smoothly
L_1	$\ w\ _1 = \sum_j w_j $	lasso	shrinks and sets many to exactly 0
both	$\rho \ w\ _1 + \frac{1-\rho}{2} \ w\ _2^2$	elastic net	sparse and stable

- $\lambda \geq 0$ is a **hyper-parameter**: 0 is least squares, ∞ predicts the mean
- The bias b is **not** penalized; **standardize first**, the penalty treats all weights alike

Ridge Regression

$$\mathcal{L}_{ridge} = \frac{1}{N} \|y - Xw - b\|^2 + \lambda \|w\|_2^2$$

$$w^* = (X^\top X + \lambda NI)^{-1} X^\top y \quad (\text{centred data})$$

- Adding λNI to $X^\top X$ makes it **invertible and well conditioned**: highly correlated polynomial features no longer blow up
- Gradient descent: the penalty adds $2\lambda w$ to the gradient, a “weight decay” towards 0
- One feature, centred: $w_{ridge} = \frac{w_{ols}}{1 + \lambda N / \sum x_i^2}$, a **shrinkage factor** below 1
- $\lambda = 0$: least squares; moderate λ : smooth and stable; $\lambda \rightarrow \infty$: $w \rightarrow 0$, the mean (more bias, less variance)

Lasso and Elastic Net

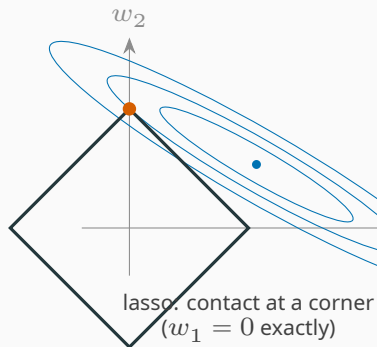
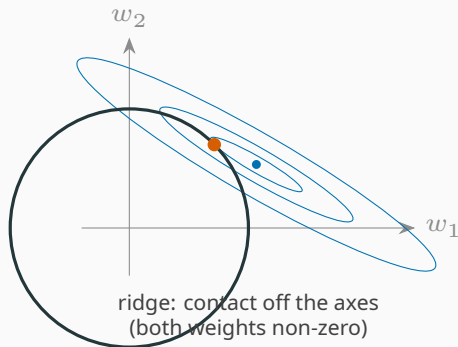
$$\mathcal{L}_{lasso} = \frac{1}{N} \|y - Xw - b\|^2 + \lambda \|w\|_1$$

- No closed form ($|w|$ has a kink at 0), but still **convex**: coordinate descent (scikit-learn) or a **proximal** step, descend on the MSE, then **soft-threshold**

$$\text{soft}(w, t) = \text{sign}(w) \max(|w| - t, 0)$$

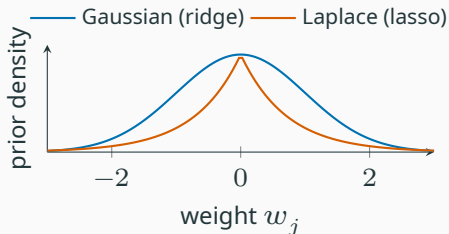
- A weight pulled less by the data than by the penalty **stays at 0: feature selection**
- `jax.grad` of $|w|$ is a *subgradient*: weights hover near 0, never exactly; the proximal step gives exact zeros (lab C3)
- **Elastic net** keeps correlated features together; lasso picks one

The Geometry: Why L_1 Gives Zeros



- Blue: contours of the squared error (centre = least squares). Black: the region $\|w\|_p \leq t$
- The solution is where the smallest contour touches the region
- The L_1 region has **corners on the axes**; contours tend to meet it there

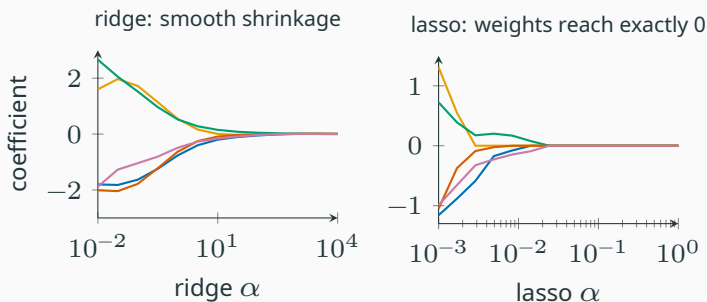
The Prior View: MAP



$$\hat{w}_{MAP} = \arg \min_w \underbrace{-\log p(y \mid X, w)}_{\text{data: MSE}} \underbrace{-\log p(w)}_{\text{prior: penalty}}$$

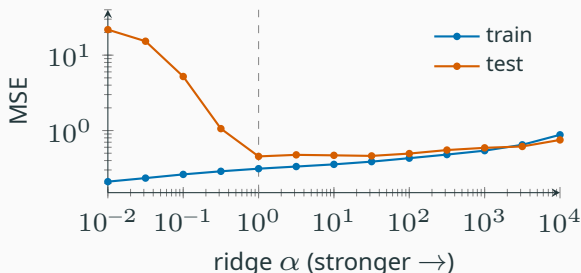
- Gaussian prior $\Rightarrow L_2$; Laplace prior $\Rightarrow L_1$ (a peak at 0: most weights are 0)
- Class 01 (MDL): the penalty is the **code length of the model**, the MSE that of the errors
- λN = noise variance / prior variance

Regularization Paths



- 300 examples, 164 degree-3 features, five coefficients; regularization is **stronger to the right** in both plots
- Ridge: every weight shrinks smoothly. Lasso: weights become exactly 0 one after the other; non-zero weights: $\alpha = 1$: 0 of 164, 0.07: 7, 0.014: 20, 0.001: 44

Choosing λ



- A U-shaped **validation** curve: too small α overfits (test 21.8), too large underfits (test 0.75); training error rises monotonically
- Choose λ on a **validation set or by cross-validation** (RidgeCV, LassoCV); the test set is touched once
- Search on a **logarithmic grid** ($10^{-4}, \dots, 10^4$); refine around the best value

Conventions: λ and scikit-learn

Model	scikit-learn minimizes	Our λ (MSE = a mean)
Ridge	$\ y - Xw\ ^2 + \alpha\ w\ ^2$	$\alpha = \lambda N$
Lasso	$\frac{1}{2N}\ y - Xw\ ^2 + \alpha\ w\ _1$	$\alpha = \lambda/2$
Logistic	$C \sum_i \ell_i + \frac{1}{2}\ w\ ^2$	$C = 1/(\lambda N)$ (loss + $\frac{\lambda}{2}\ w\ ^2$)

- The same idea, three scalings: a mismatch is the usual reason two “identical” models disagree
- Lab: after mapping λ , our ridge equals Ridge to $3 \cdot 10^{-16}$ and our logistic equals LogisticRegression to $2 \cdot 10^{-6}$
- Lasso weights differ from scikit-learn’s by up to 0.15–0.20 while the **objective differs by** 10^{-5} : nearly collinear features have many equally good solutions

Result: Taming a Degree-3 Model

Model (164 features, 300 examples)	Test MSE	Non-zero weights
plain least squares	159.8	164
ridge ($\alpha = 1$, best on test)	0.454	164
ridge, α by 5-fold CV (31.6)	0.461	164
lasso ($\alpha = 0.0203$, best on test)	0.494	15
lasso, α by 5-fold CV (0.01)	0.547	19
elastic net ($\rho = 0.5$)	0.535	39
<i>(for reference) the plain line, 8 features</i>	<i>0.467</i>	<i>8</i>

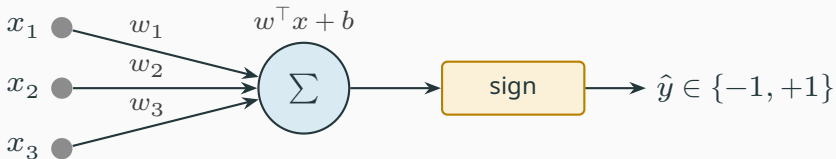
- Regularization turns a disastrous model into a good one; the lasso does it with **15** of 164 features
- Choosing α on the test set (the “best” rows) is optimistic: the CV rows are the honest numbers

Live Demo R5: Ridge, Lasso, Elastic Net

- Notebook `01_linear_regression.ipynb`, section **R5**
- Constraint geometry; ridge and lasso paths; the validation curve; RidgeCV and LassoCV
- Question: the plain line (0.467) still beats the ridge at 300 examples: when would the degree-3 model win?

The Perceptron

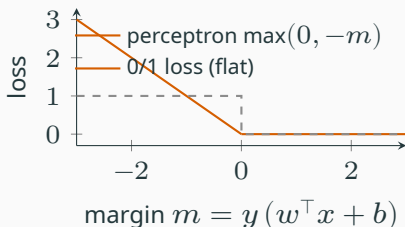
A Neuron That Draws a Line



$$\hat{y} = \text{sign}(w^\top x + b) \quad y \in \{-1, +1\}$$

- The first learning machine (Rosenblatt, 1958): a weighted vote, then a hard decision
- The **decision boundary** $w^\top x + b = 0$ is a hyperplane; w is its normal, pointing to the $+1$ side
- The **margin** $m = y (w^\top x + b)$ is positive when the point is correctly classified
- It is the building block of neural networks (Class 05)

The Perceptron Loss and Its Update



$$\ell = \max(0, -m), \quad \nabla_w \ell = \begin{cases} -y x & m \leq 0 \text{ (a mistake)} \\ 0 & m > 0 \end{cases}$$

- A gradient step of length 1: on a mistake, $w \leftarrow w + y x$ and $b \leftarrow b + y$; otherwise nothing
- **The perceptron rule is a subgradient step** on this loss: in the lab, `jax.grad` of one example *is* the update

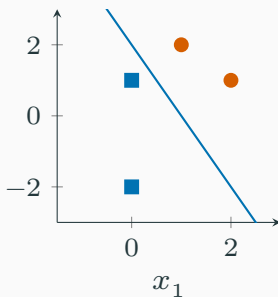
Worked Example: the Perceptron by Hand

Points

$(2, 1)^+$, $(1, 2)^+$, $(0, -2)^-$, $(0, 1)^-$;

$w = (0, 0)$, $b = 0$

epoch	point	y	score	new w	new b
1	$(2, 1)$	+	0	$(2, 1)$	1
1	$(0, 1)$	-	2	$(2, 0)$	0
2	$(0, -2)$	-	0	$(2, 2)$	-1
2	$(0, 1)$	-	1	$(2, 1)$	-2
3	all four			no change	



- Only the mistakes are listed (a **score** of 0 counts as a mistake); the other visits change nothing
- Final boundary $2x_1 + x_2 - 2 = 0$: all four points are correct, so the algorithm stops

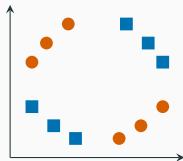
Convergence, and When It Fails

Data	Mistakes per epoch	Outcome
separable blobs	2, 0, 0, 0, ...	converges
overlapping blobs	20, 14, 14, 14, 11, 14, 10, 14, ...	never stops
XOR, raw features	98, 100, 98, 92, 96, ...	accuracy 0.42
XOR, degree-2 features	16, 2, 0, 0, ...	converges

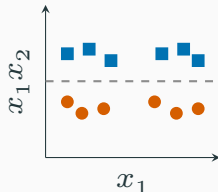
- **Novikoff (1962):** on linearly separable data with margin γ and $\|x\| \leq R$, at most $(R/\gamma)^2$ updates, whatever the order
- On overlapping data it cycles: the final weights are arbitrary, and there is no “best” line in sight
- The exact behaviour is shared with scikit-learn: our `jax.grad_perceptron` and `Perceptron(penalty=None, eta0=1, shuffle=False)` give **identical** weights (difference **0.0**)

XOR: Beyond a Straight Line

raw (x_1, x_2) : no line works



with $x_1 x_2$: a threshold works



- XOR: class 1 when the coordinates have **different signs** (Minsky and Papert, 1969)
- Raw features: chance level (accuracy **0.42**); adding $x_1 x_2$ makes it **separable**
- Same for logistic regression: accuracy 0.655 on raw XOR, **1.0** with degree-2 features (lab)
- Learning the features instead is the hidden layer (Class 05)

The Perceptron on Spam

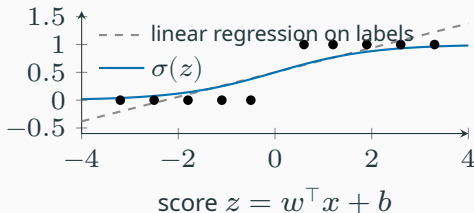
- One pass over the 3680 training e-mails (log frequencies, standardized), a step of 1:
 - **394 mistakes** in the epoch (10.7%)
 - test accuracy **0.912**
- Fast, online, no probabilities, no learning rate to tune
- Sensitive to the **order** of the examples and to feature scale (a large feature dominates the update)
- No notion of *how confident* a prediction is: a point just inside the boundary looks the same as one far away
- Its smoother cousin, next: **logistic regression**, which turns the same score $w^\top x + b$ into a probability

Live Demo C1: The Perceptron

- Notebook `02_perceptron_logistic.ipynb`, section **C1**
- The update as `jax.grad` of one example; boundary snapshots after each epoch
- Separable, overlapping and XOR data; polynomial features fix XOR
- Same weights as scikit-learn's Perceptron

Logistic Regression

From a Score to a Probability



$$p(y = 1 \mid x) = \sigma(w^T x + b), \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$

- A line fitted to 0/1 labels leaves $[0, 1]$: not a probability (schematic data)
- The sigmoid maps any score into $(0, 1)$; the boundary $p = 0.5$ is still $w^T x + b = 0$; the **threshold** is a choice

Odds and Log-Odds

p	odds $\frac{p}{1-p}$	log-odds z
0.01	0.010	-4.60
0.10	0.111	-2.20
0.50	1	0
0.90	9	2.20
0.99	99	4.60

$$\log \frac{p}{1-p} = w^{\top} x + b$$

- The model is **linear in the log-odds**: each weight adds evidence for or against spam
- e^{w_j} is the **odds ratio**: the odds get multiplied by e^{w_j} when feature j grows by one standard deviation
- Spam: char_freq_! +0.74 ($\times 2.09$). Ham: word_freq_hp -1.37 ($\times 0.25$)

Cross-Entropy: Maximum Likelihood for Labels

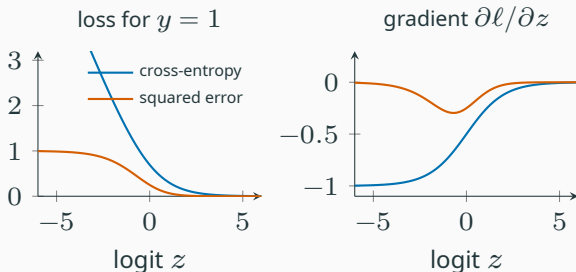
Each label is a Bernoulli variable with $P(y = 1) = p_i$; the likelihood of the data and its negative log:

$$-\log \prod_i p_i^{y_i} (1-p_i)^{1-y_i} = \sum_i \ell_i, \quad \ell_i = -y_i \log p_i - (1-y_i) \log (1-p_i)$$

$$\ell_i = \log(1 + e^{z_i}) - y_i z_i \quad (z_i = w^\top x_i + b)$$

- The **binary cross-entropy**: the code length of the labels (Class 01)
- With the logit z and `jnp.logaddexp(0, z)` it never overflows
- Confident and right: $\ell \approx 0$; confident and **wrong**: ℓ grows like $|z|$

Why Not the Squared Error?



- For a confidently **wrong** prediction ($z \ll 0$) the cross-entropy gradient is -1 : a strong push
- The squared error's gradient **vanishes** there: the model is stuck exactly when it is most wrong
- The cross-entropy is **convex** in (w, b) ; the squared error on a sigmoid is not

The Gradient Is a Residual Again

$$\nabla_w \mathcal{L} = \frac{1}{N} X^\top (p - y), \quad \frac{\partial \mathcal{L}}{\partial b} = \frac{1}{N} \sum_i (p_i - y_i)$$

Worked example: one e-mail, feature $x = 1.5$, weights $w = 1.2$, $b = -0.5$

	$y = 1$ (spam)	$y = 0$ (ham)
score $z = 1.2 \cdot 1.5 - 0.5$	1.300	1.300
probability $p = \sigma(z)$	0.786	0.786
loss $-\log p_y$	0.241	1.541
$\partial \ell / \partial w = (p - y) x$	-0.321	+1.179
$\partial \ell / \partial b = p - y$	-0.214	+0.786

- Like linear regression: (prediction — target) $\times$ feature.
Not typed in the lab: `jax.grad` produces it

Training It: Convex but No Closed Form

- $\nabla \mathcal{L} = 0$ is **nonlinear** in w : no closed form. The loss is convex: GD, Adam or L-BFGS reach the global minimum
- Newton's method on it is IRLS (iteratively reweighted least squares)
- **Separable data**: the loss keeps decreasing as $\|w\| \rightarrow \infty$. **Regularization fixes it**

$$\mathcal{L}_\lambda = \frac{1}{N} \sum_i \ell_i + \frac{\lambda}{2} \|w\|^2, \quad C = \frac{1}{\lambda N}$$

- Spam ($\lambda = 10^{-3}$): our weights equal scikit-learn's to $2 \cdot 10^{-6}$

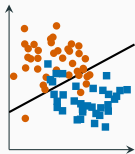
Result: Spam Detection

Model	Accuracy	Precision	Recall	F_1	MCC
majority class ("all ham")	0.606	0	0	0	0
perceptron, 1 epoch	0.912	0.875	0.906	0.890	0.817
logistic regression ($C = 0.1$)	0.937	0.913	0.928	0.921	0.869

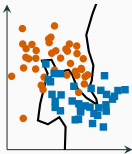
- Test set: 921 e-mails (363 spam). Log frequencies, standardized with the training statistics
- The logistic model is better than a single pass of the perceptron, *and* it outputs a probability per e-mail
- 92.8% of the spam is caught; **8.7%** of the e-mails flagged as spam are actually ham: the price of the default threshold of 0.5
- Next: what the numbers mean, and how to choose the threshold

Non-Linear Boundaries: Features and Regularization

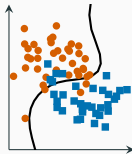
degree 1
train 0.84, test 0.848



degree 15, $C = 10^6$
train 0.955, test 0.859



degree 15, $C = 0.1$
train 0.860, test 0.876



- “Two moons”, 200 noisy training points (a sample is drawn), 2000 test points; polynomial features + logistic regression
- Unregularized degree 15: an intricate boundary that fits the noise (train 0.955, test 0.859); with $C = 0.1$ it stays smooth and **generalizes better** (test 0.876)
- The best unregularized model here is degree 3 (test 0.887): flexibility and constraint both need tuning

More Than Two Classes: Softmax

$$p(y = k \mid x) = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}, \quad z_k = w_k^\top x + b_k$$

- One score per class; the **softmax** turns them into probabilities that sum to 1
- The loss is the same cross-entropy: $-\log p(y_i \mid x_i)$; for $K = 2$ it reduces to the sigmoid
- LogisticRegression uses it for multi-class problems; every neural-network classifier ends with it (Class 05)
- The perceptron extends to K classes by keeping one weight vector per class and updating the true and the predicted class

Live Demo C2: Logistic Regression

- Notebook `02_perceptron_logistic.ipynb`, section **C2**
- The sigmoid, and cross-entropy versus squared error (loss and gradient)
- Training with `jax.value_and_grad`; weights and odds ratios read off as words (hp, george for ham; !, remove for spam)
- Section **C4**: polynomial features and the regularization strength on “two moons”

Classification Metrics

The Confusion Matrix

		predicted	
		spam	ham
actual	spam	TP = 337 spam caught	FN = 26 spam missed
	ham	FP = 32 ham hidden	TN = 526 ham delivered

921 test e-mails
363 spam, 558 ham
(positive class: spam)

- Every metric of this section is a function of these **four counts** (the logistic model at threshold 0.5)
- The two errors are **not** equally bad: a false positive hides a real e-mail, a false negative shows one spam
- Multi-class: an $K \times K$ matrix; the diagonal is correct

Accuracy, Precision and Recall

$$\text{accuracy} = \frac{TP + TN}{n} = \frac{337 + 526}{921} = 0.937$$

$$\text{precision} = \frac{TP}{TP + FP} = \frac{337}{369} = 0.913$$

$$\text{recall} = \frac{TP}{TP + FN} = \frac{337}{363} = 0.928$$

- **Accuracy:** the fraction of correct decisions; misleading when the classes are imbalanced
- **Precision:** of the e-mails flagged as spam, how many really are? (cost of a false positive)
- **Recall** (sensitivity, true-positive rate): of the spam, how much was caught? (cost of a false negative)

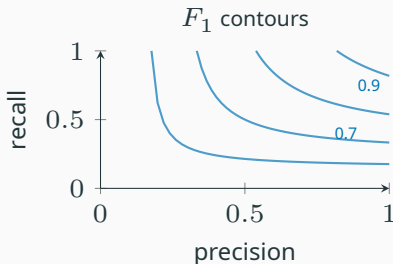
The Accuracy Paradox

Test set: all 558 ham + 11 spam (1.9% spam)	Accuracy	Precision	Recall	F_1	MCC
"everything is ham"	0.981	0	0	0	0
our classifier	0.942	0.238	0.909	0.377	0.449

- The useless filter has the **higher accuracy**: it never fails on the 98% that are ham
- Our classifier finds **10 of 11** spam, at the price of 32 false alarms (precision 0.238)
- Class 01: "the metric must match the goal". With rare positives, look at precision, recall and MCC, never accuracy alone
- Also: always compare with the **majority-class baseline** (0.606 on the balanced-ish spam data)

F_1 : One Number for Precision and Recall

$$F_1 = \frac{2PR}{P + R} = \frac{2TP}{2TP + FP + FN} = \frac{674}{732} = 0.921$$



- The **harmonic mean**: low if *either* is low (precision 1, recall 0.01 gives $F_1 = 0.02$, not 0.5)
- Ignores the **true negatives**: F_1 can look fine for a model that is useless on the negative class

MCC: the Whole Matrix in One Number

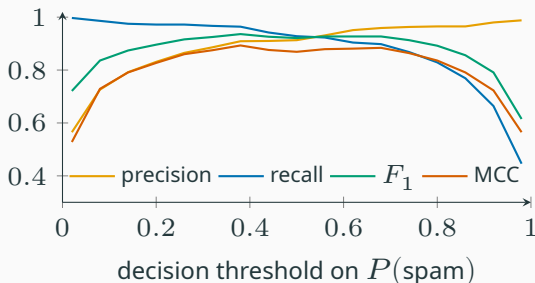
$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}$$

$$\text{Ours: } \frac{337 \cdot 526 - 32 \cdot 26}{\sqrt{369 \cdot 363 \cdot 558 \cdot 552}} = \frac{176\,430}{203\,120} = 0.869$$

- The correlation of predicted and actual labels: +1 perfect, 0 chance, -1 total disagreement
- Uses all four cells: **symmetric** in the classes, robust to imbalance

"Everything is spam" (test set, 39.4% spam)	Value
accuracy	0.394
precision / recall	0.394 / 1.000
F_1	0.565
MCC	0.000

The Threshold Is a Decision



- Raising the threshold: fewer e-mails flagged, **precision up, recall down**. Lowering it: the reverse
- F_1 and MCC peak around 0.4–0.7; the best threshold for *our costs* may be far from it

Choosing the Threshold

Threshold	Precision	Recall	F_1	MCC	Ham hidden
0.50	0.913	0.928	0.921	0.869	32
0.90	0.973	0.705	0.818	0.749	7
0.99	0.993	0.386	0.556	0.521	1

- If hiding a real e-mail costs, say, 20 times a missed spam, the third row is the sensible one, at the price of **61% of the spam** getting through
- **Choose the threshold on validation data** with the costs in hand; the test set only reports the result (in the lab: precision 0.99 on the *training* data gives 0.974 on test)
- Threshold-free summaries: the **ROC curve** (recall against false-positive rate) and the **precision-recall curve**; AUC of ours: **0.981**
- Prefer PR curves when positives are rare

Which Metric When?

Situation	Look at	Why
balanced classes, equal costs rare positives	accuracy, MCC precision, recall, PR curve	simple, all cells accuracy is inflated by negatives
a missed positive is costly (screening)	recall	do not miss sick patients
a false alarm is costly (spam filter)	precision	do not hide real e-mails
one number, any imbalance ranking quality, threshold not fixed	MCC ROC AUC, average precision	uses TP, TN, FP, FN independent of the threshold

- Regression counterpart: MSE/RMSE for quadratic costs, MAE for robust, sMAPE for relative, R^2 against the mean
- Whatever the metric: baseline, spread over folds, and a **test set used once**

Live Demo C3: Classification Metrics

- Notebook `02_perceptron_logistic.ipynb`, section **C3**
- The confusion matrix and all five metrics, checked against scikit-learn
- The imbalanced test set (1.9% spam): the accuracy paradox live
- The threshold sweep, the precision-recall curve, the ROC AUC
- Question: which threshold would you ship for a personal mailbox? For a company? # Summary

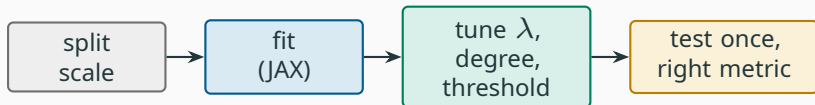
The Models Side by Side

	Linear regression	Perceptron	Logistic regression
Output	a number	a class	a probability
Loss	squared error	$\max(0, -m)$	cross-entropy
Training	closed form or GD	online updates	GD (convex)
Boundary	(hyperplane fit)	linear	linear
Assumes	linear mean, Gaussian noise	separability	linear log-odds
Result	$R^2 = 0.681$ (housing)	accuracy 0.912 (spam)	accuracy 0.937 (spam)

- Regularization (L_2, L_1) applies to every model that has weights; polynomial features make any of them non-linear
- Next class: where the losses come from (**maximum likelihood**) and a different kind of classifier (**Naive Bayes**)

Choosing and Checking

- **Start simple:** a baseline, then a linear model; add flexibility only if validation asks
- **A probability, not just a class:** logistic regression
- **Many features, few examples:** regularize; lasso to select features
- **Never trust a number without:** a baseline, a spread, a test set used once, the right metric
- **Watch for:** unscaled features, leakage, imbalance, a threshold left at 0.5



Key Takeaways

- **A model is (hypothesis, loss, optimizer):** write the loss in `jax.numpy`, `jax.grad` does the rest
- **Linear regression:** MSE, closed form or GD; **polynomial features** add flexibility (and overfitting)
- **Regularization:** ridge shrinks, lasso selects; λ is chosen on validation data
- **Perceptron:** a subgradient step. **Logistic regression:** cross-entropy, probabilities
- **Metrics:** MSE, MAE, sMAPE, R^2 ; confusion matrix, precision, recall, F_1 , **MCC**; the threshold is a decision
- **Protocol:** split, scale on training, tune by CV, test once, always a baseline

Lab 03: Build It in JAX

Part	Task	sklearn twin
A	splits, scaling, metrics from scratch	metrics, model_selection
B	linear regression: MSE, <code>jax.grad</code> , normal equation	LinearRegression
C	polynomial features, ridge, lasso, proximal step	Ridge, Lasso
D	perceptron: its update is a <code>jax.grad</code>	Perceptron
E	logistic regression, thresholds	LogisticRegression
F	challenge: one protocol, three models	cross_val_score

- Guide `practice/03_linear_models.pdf`, notebook `lab03_linear_models.ipynb`, solution in `solutions/`
- Every model is checked against its scikit-learn twin (a printed maximum difference)
- **Project 1** is released today: start with this class's protocol

Next Class: Probabilistic Models

- **Where do the losses come from?** Maximum likelihood: MSE is Gaussian noise, cross-entropy is Bernoulli labels
- **MAP:** a prior on the parameters; ridge is a Gaussian prior, Laplace smoothing a Beta prior
- **Naive Bayes:** Bayes' rule and independence; discrete and continuous features, for classes **and** numbers, trained by counting
- **Probabilities you can trust:** calibration, log-loss, and decisions with costs

References (1/2)

- G. James, D. Witten, T. Hastie, R. Tibshirani, J. Taylor, *An Introduction to Statistical Learning with Applications in Python*, 2023: linear models, regularization, classification
- C. Bishop, *Pattern Recognition and Machine Learning*, 2006: chapters 3 and 4 (linear models)

References (2/2)

- R. Tibshirani, "Regression shrinkage and selection via the lasso", *JRSS B*, 1996
- H. Zou, T. Hastie, "Regularization and variable selection via the elastic net", *JRSS B*, 2005
- D. Chicco, G. Jurman, "The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation", *BMC Genomics*, 2020
- F. Rosenblatt, "The perceptron: a probabilistic model for information storage and organization in the brain", *Psychological Review*, 1958
- M. Minsky, S. Papert, *Perceptrons*, 1969